

# OPERATING SYSTEMS ASSIGNMENT 2

## Multithreaded Perfect Number Checker

**Name:** Paul Biju Cheeramvelil

**Roll No:** B240266CS

**Date:**03/04/2026

---

### 1. Problem Statement

Design and implement a multithreaded program in C to determine whether a given number **N** is a perfect number using **P threads**. A perfect number is defined as a number whose sum of all its proper divisors (excluding itself) is equal to the number.

### 2. Approach

The program uses multithreading to speed up the computation of factors of **N**.

- Instead of checking all numbers from 1 to **N**, the search space is reduced to 1 to  $\sqrt{N}$  for efficiency.
- The range is divided among **P threads** such that each thread processes a unique subset.
- Each thread checks whether numbers in its range divide **N**.
- If a divisor is found:
  - The divisor and its corresponding pair ( $N/i$ ) are added to a shared sum.
- A **mutex lock** is used to ensure safe updates to the shared variable.
- After all threads complete, the final sum is compared with **N** to determine whether it is a perfect number.

### 3. Work Partitioning

- Total range: 1 to  $\sqrt{N}$
- Each thread is assigned a subrange of this interval.
- The size of each subrange is:  
chunk =  $\sqrt{N} / P$
- The last thread handles any remaining values to ensure complete coverage.

## 4. Synchronization

- The variable `sum` is shared among all threads.
- To prevent race conditions, a **mutex** (`pthread_mutex_t`) is used.
- Each thread locks the mutex before updating `sum` and unlocks it afterward.
- This guarantees correct and consistent results.

## 5. Algorithm

1. Read input values **N** and **P** from command line.
2. If **N**  $\leq 1$ , report that it is not a perfect number.
3. Compute `limit =  $\sqrt{N}$` .
4. Adjust number of threads if **P**  $>$  `limit`.
5. Divide the range among threads.
6. Create **P** threads.
7. Each thread:
  - Iterates through its assigned range
  - Checks divisibility
  - Updates shared sum using mutex
8. Wait for all threads to complete using `pthread_join`.
9. Compare `sum` with **N**.
10. Print whether the number is perfect or not.

## 6. Observations

### Case 1: Perfect Number (N = 28)

- Factors: 1, 2, 4, 7, 14
- Sum = 28
- Output: Perfect number

### Case 2: Non-Perfect Number (N = 10)

- Factors: 1, 2, 5
- Sum = 8
- Output: Not a perfect number

### Case 3: Larger Input (N = 496)

- Correctly identified as a perfect number
- Execution is faster due to:

- Multithreading
- Reduced search space using  $\sqrt{N}$

## 7. Results

| N   | P | Result             |
|-----|---|--------------------|
| 6   | 2 | Perfect Number     |
| 28  | 4 | Perfect Number     |
| 496 | 4 | Perfect Number     |
| 10  | 3 | Not Perfect Number |
| 15  | 2 | Not Perfect Number |

## 8. Conclusion

The multithreaded approach efficiently determines whether a number is perfect by distributing the computation across multiple threads. The use of mutex ensures correct synchronization and prevents race conditions. Additionally, limiting the search space to  $\sqrt{N}$  significantly improves performance. The program successfully handles both perfect and non-perfect numbers and demonstrates correct parallel execution.